

Assessing Credit Risk with Machine

Learning: A Case Study Based on Real Banking Data

Student Name: Elaa Benabdallah

Student Group: [BA IT]

Tunis Business School Date: February 5th, 2026

Introduction

Credit lending is one of the most important decisions in banking: it generates revenue through interest and fees, but it also exposes the institution to losses when borrowers fail to repay. As the number of applications increases and customer profiles become more diverse, relying only on manual assessment becomes difficult to scale and may lead to inconsistent decisions. For this reason, banks increasingly use **data-driven decision support** to make credit evaluation faster, more consistent, and more objective.

In this project, we work with the **German Credit (Statlog)** dataset, a well-known benchmark for credit risk modeling. The dataset contains information about **1,000 loan applicants**, described by **20 explanatory variables** (financial and demographic attributes), and a target variable indicating whether the applicant was a **Good** or **Bad** credit risk based on repayment behavior. The objective is to build a classification model that can predict the credit risk category for new applicants. We apply the **K-Nearest Neighbors (KNN)** algorithm, a similarity-based method that classifies a new applicant by looking at the outcomes of the most similar applicants in historical data.

Problem Statement

Problem to solve:

Given an applicant's characteristics (e.g., credit amount, loan duration, credit history, employment status, etc.), predict whether the applicant is a **Good** or **Bad** credit risk.

Business goal:

Support the bank's approval decision by reducing losses due to default and improving the quality of credit screening. Importantly, classification errors do not have the same impact:

- **Most costly error (False Positive, FP):** predicting **Good** when the true class is **Bad** → the bank approves a risky applicant, which can lead to default and financial loss.
- **Less costly error (False Negative, FN):** predicting **Bad** when the true class is **Good** → the bank rejects a reliable applicant, resulting mainly in lost business opportunity.

Machine learning objective:

Build a KNN classifier with an appropriate choice of **k**, and evaluate performance using metrics that reflect the real objective: not only overall accuracy, but also the model's ability to detect **Bad** applicants (e.g., recall/sensitivity for the Bad class) and, when possible, a **cost-sensitive evaluation** that penalizes FN errors more heavily than FP errors.

Understanding the Data

A11	6	A34	A43	1169	A65	A75	4	A93	A101	4	A121	67	A143	A152	2	A173	1	A192	A201	1
A12	48	A32	A43	5951	A61	A73	2	A92	A101	2	A121	22	A143	A152	1	A173	1	A191	A201	2
A14	12	A34	A46	2096	A61	A74	2	A93	A101	3	A121	49	A143	A152	1	A172	2	A191	A201	1
A11	42	A32	A42	7882	A61	A74	2	A93	A103	4	A122	45	A143	A153	1	A173	2	A191	A201	1
A11	24	A33	A40	4870	A61	A73	3	A93	A101	4	A124	53	A143	A153	2	A173	2	A191	A201	2
A14	36	A32	A46	9055	A65	A73	2	A93	A101	4	A124	35	A143	A153	1	A172	2	A192	A201	1
A14	24	A32	A42	2835	A63	A75	3	A93	A101	4	A122	53	A143	A152	1	A173	1	A191	A201	1
A12	36	A32	A41	6948	A61	A73	2	A93	A101	2	A123	35	A143	A151	1	A174	1	A192	A201	1

Table of attributes (number, type, description, and coding)

Attribute #	Type	Description	Codes / Meaning
1	Qualitative	Status of existing checking account	A11 : ... < 0 DM A12 : 0 <= ... < 200 DM A13 : ... >= 200 DM / salary assignments for at least 1 year A14 : no checking account
2	Numerical	Duration in month	Numeric (continuous or integer values as recorded).
3	Qualitative	Credit history	A30 : no credits taken/ all credits paid back duly A31 : all credits at this bank paid back duly A32 : existing credits paid back duly till now A33 : delay in paying off in the past A34 : critical account/ other credits existing (not at this bank)
4	Qualitative	Purpose	A40 : car (new) A41 : car (used)

			A42 : furniture/equipment A43 : radio/television A44 : domestic appliances A45 : repairs A46 : education A47 : (vacation - does not exist?) A48 : retraining A49 : business A410 : others
5	Numerical	Credit amount	Numeric (continuous or integer values as recorded).
6	Qualitative	Savings account/bonds	A61 : ... < 100 DM A62 : 100 <= ... < 500 DM A63 : 500 <= ... < 1000 DM A64 : .. >= 1000 DM A65 : unknown/ no savings account
7	Qualitative	Present employment since	A71 : unemployed A72 : ... < 1 year A73 : 1 <= ... < 4 years A74 : 4 <= ... < 7 years A75 : .. >= 7 years
8	Numerical	Installment rate in percentage of disposable income	Numeric (continuous or integer values as recorded).
9	Qualitative	Personal status and sex	A91 : male : divorced/separated A92 : female : divorced/separated/ma rried A93 : male : single A94 : male : married/widowed A95 : female : single

10	Qualitative	Other debtors / guarantors	A101 : none A102 : co-applicant A103 : guarantor
11	Numerical	Present residence since	Numeric (continuous or integer values as recorded).
12	Qualitative	Property	A121 : real estate A122 : if not A121 : building society savings agreement/ life insurance A123 : if not A121/A122 : car or other, not in attribute 6 A124 : unknown / no property
13	Numerical	Age in years	Numeric (continuous or integer values as recorded).
14	Qualitative	Other installment plans	A141 : bank A142 : stores A143 : none
15	Qualitative	Housing	A151 : rent A152 : own A153 : for free
16	Numerical	Number of existing credits at this bank	Numeric (continuous or integer values as recorded).
17	Qualitative	Job	A171 : unemployed/ unskilled - non-resident A172 : unskilled - resident A173 : skilled employee / official A174 : management/ self-employed/ highly qualified employee/ officer
18	Numerical	Number of people being liable to provide maintenance for	Numeric (continuous or integer values as recorded).

19	Qualitative	Telephone	A191 : none A192 : yes, registered under the customers name
20	Qualitative	foreign worker	A201 : yes A202 : no
21	Numerical	Good or Bad	1: Good 2: Bad

R Code

Part 1 — Data preparation & Exploratory Data Analysis (EDA)

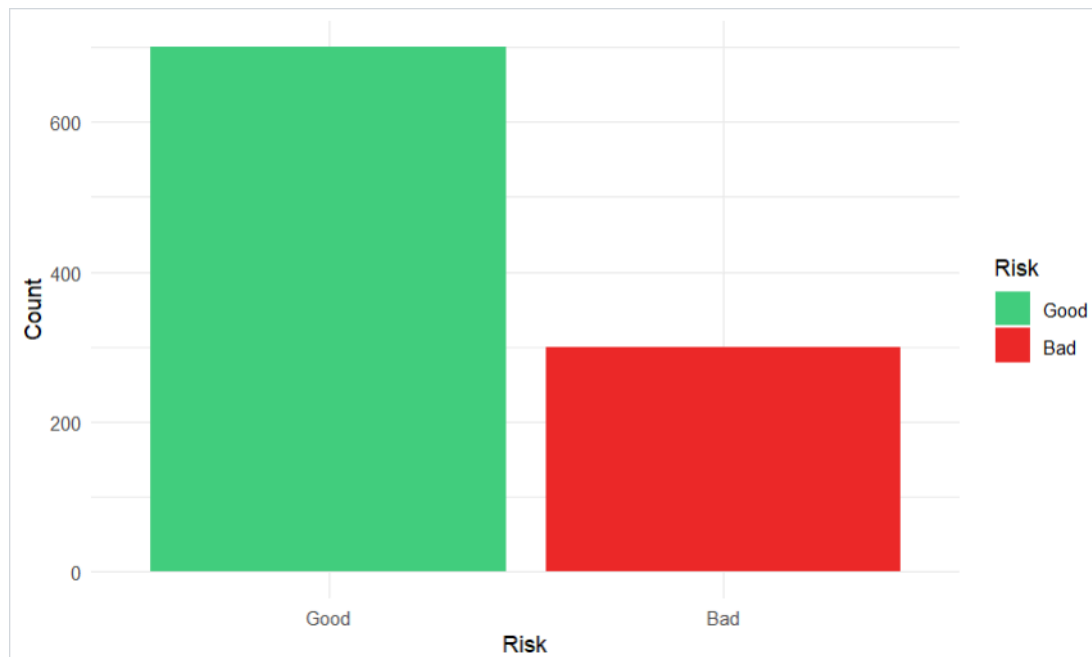
```
> setwd("C:/Users/USER/Downloads/statlog+german+credit+data")
> list.files()
[1] "german.data"          "german.data-numeric" "german.doc"          "Index"
> df <- read.table("german.data", header = FALSE, stringsAsFactors = FALSE)
> dim(df) # should be 1000 21
[1] 1000 21
> colnames(df) <- c(
+   "CheckingStatus", "Duration", "CreditHistory", "Purpose", "CreditAmount",
+   "Savings", "EmploymentSince", "InstallmentRate", "PersonalStatusSex",
+   "OtherDebtors", "ResidenceSince", "Property", "Age", "OtherInstallmentPlans",
+   "Housing", "ExistingCredits", "Job", "Dependents", "Telephone", "ForeignWorker",
+   "Risk"
+ )
> names(df)
[1] "CheckingStatus"      "Duration"            "CreditHistory"
[4] "Purpose"            "CreditAmount"       "Savings"
[7] "EmploymentSince"    "InstallmentRate"    "PersonalStatusSex"
[10] "OtherDebtors"       "ResidenceSince"     "Property"
[13] "Age"                "OtherInstallmentPlans" "Housing"
[16] "ExistingCredits"    "Job"                "Dependents"
[19] "Telephone"         "ForeignWorker"      "Risk"
> head(df$Risk)
[1] 1 2 1 1 2 1
> df$Risk <- as.integer(df$Risk)
> df$Risk <- factor(ifelse(df$Risk == 1, "Good", "Bad"),
+   levels = c("Good", "Bad"))
> table(df$Risk)

Good  Bad
  700   300
> pkgs <- c("tidyverse", "rsample", "recipes", "caret", "pROC")
> list.files()
[1] "german.data"          "german.data-numeric" "german.doc"          "Index"
```

Chart 1 — Risk distribution (bar plot)

```
library(ggplot2)|

ggplot(df, aes(x = Risk, fill = Risk)) +
  geom_bar() +
  scale_fill_manual(values = c("Good" = "seagreen3", "Bad" = "firebrick2")) +
  labs(x = "Risk", y = "Count") +
  theme_minimal()
```



The bar plot confirms the **70/30** class split. This imbalance explains why a classifier may be biased toward predicting “Good”, so we must pay special attention to **False Negatives** (Bad predicted as Good), which are costly in credit risk.

```
ggplot(df, aes(x = factor(InstallmentRate), fill = Risk)) +
  geom_bar(position = "dodge") +
  labs(title = "Installment Rate by Risk",
       x = "Installment rate category",
       y = "Count") +
  theme_minimal()
```

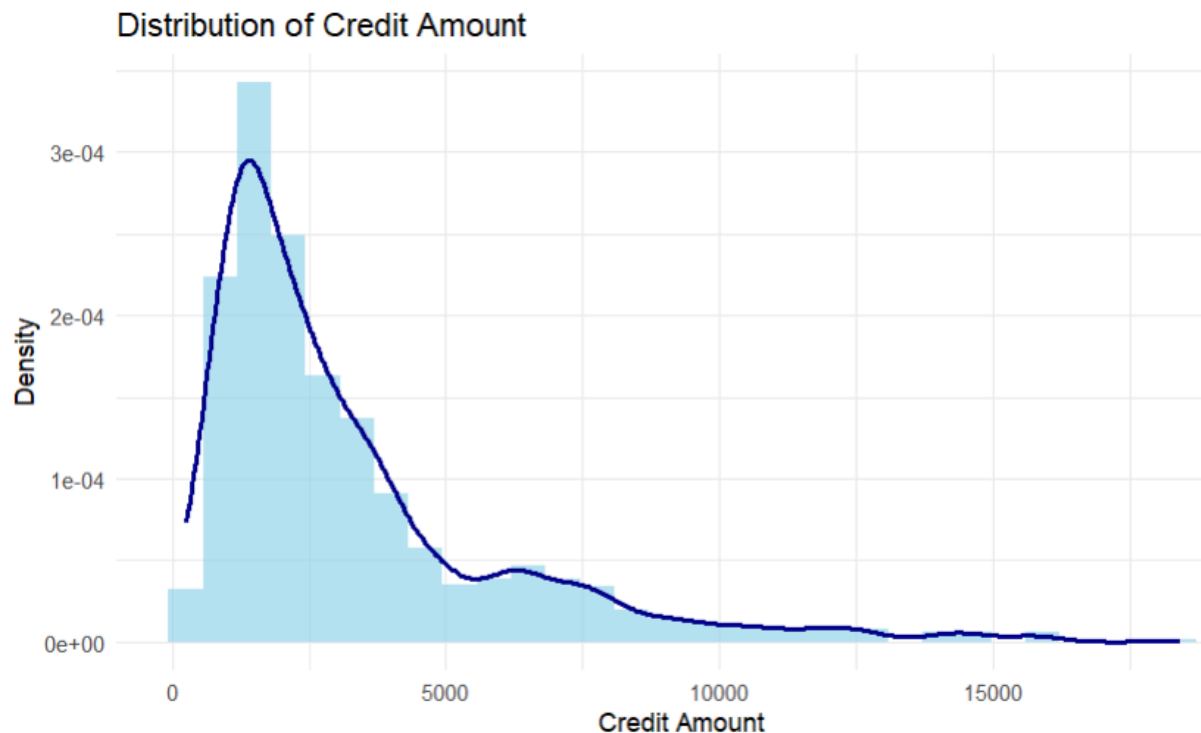
Main observations

1. **Category 4 has the highest number of applicants** for both classes.
This means most people in this dataset fall into installment-rate category 4.

2. In **every category**, the **Good bar is higher than the Bad bar**.
That matches your dataset being **70% Good / 30% Bad** overall.
3. The **number of Bad cases increases in category 4** (the blue bar is largest there).
So, more “Bad” borrowers are present among people in category 4 *in absolute count*.

Chart 2 — Distribution of Credit Amount (histogram + density)

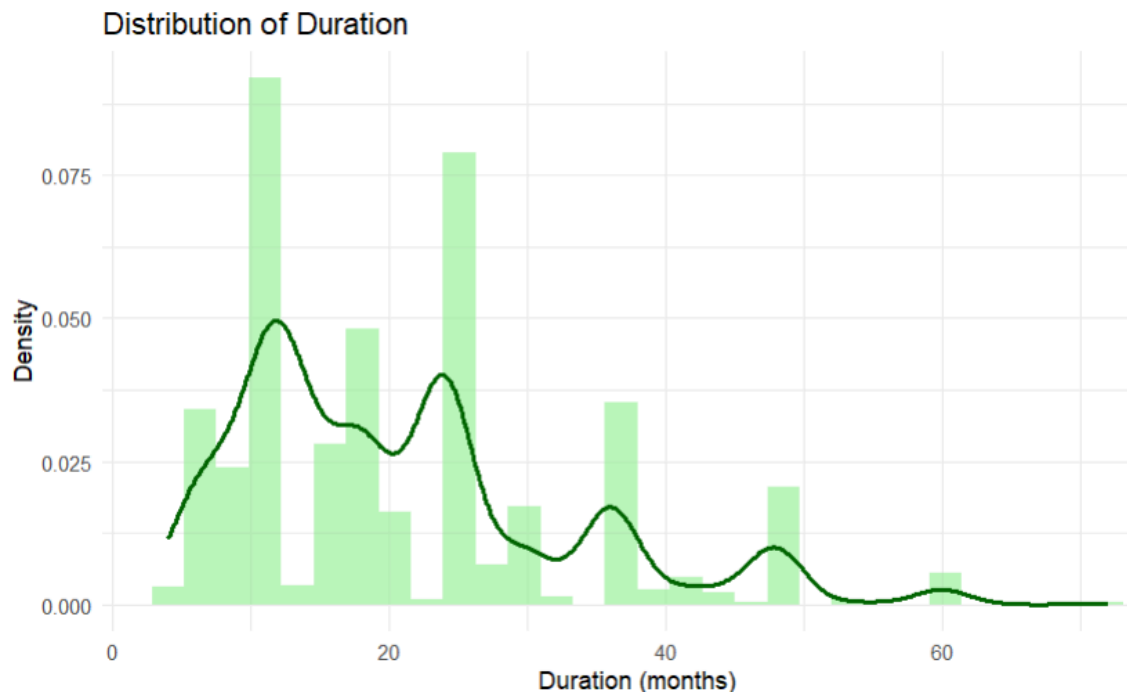
```
ggplot(df, aes(x = CreditAmount)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = "skyblue", alpha = 0.6) +
  geom_density(color = "darkblue", size = 1) +
  labs(title = "Distribution of Credit Amount",
       x = "Credit Amount",
       y = "Density") +
  theme_minimal()
```



Credit amount is strongly right-skewed: most loans are small to medium, while a few are very large (outliers). Because KNN is distance-based, this wide range can dominate distance calculations, which is why normalization/standardization is necessary before training KNN.

Chart 3 — Distribution of Duration (histogram + density)

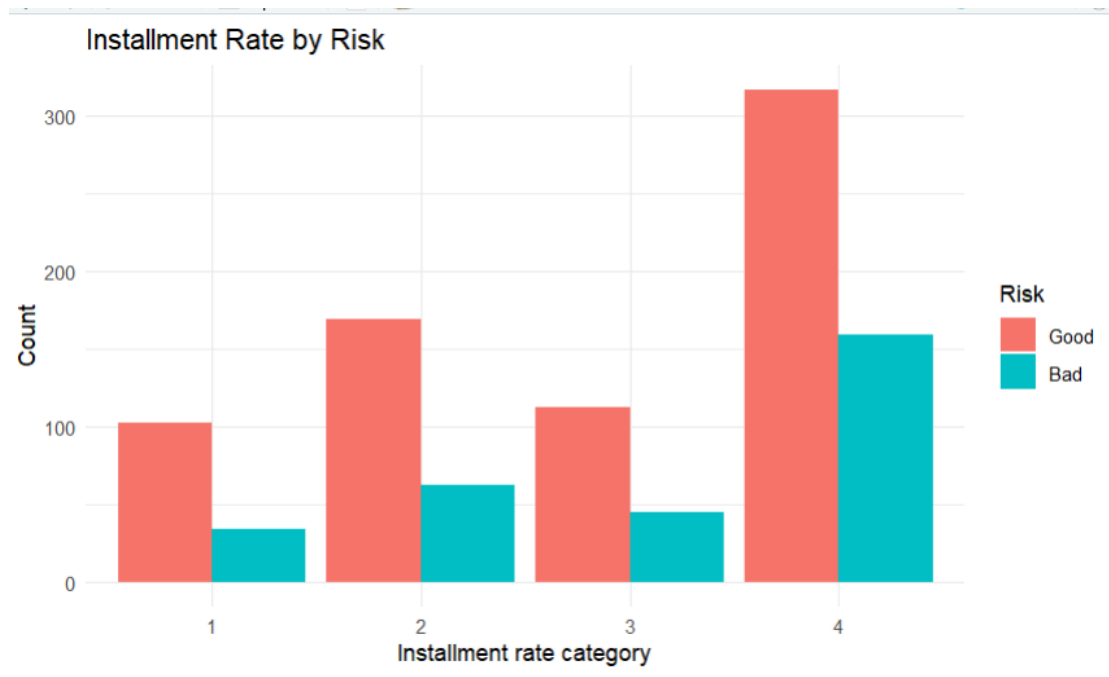
```
ggplot(df, aes(x = Duration)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = "lightgreen", alpha = 0.6) +
  geom_density(color = "darkgreen", size = 1) +
  labs(title = "Distribution of Duration",
       x = "Duration (months)",
       y = "Density") +
  theme_minimal()
```



Duration is concentrated around standard contract lengths (peaks near 12 and 24 months), and long durations are less frequent. The variable spans a wide numeric range, so it must be scaled to avoid dominating distances in KNN.

Chart 4 — Installment Rate by Risk (grouped bar plot)

```
ggplot(df, aes(x = factor(InstallmentRate), fill = Risk)) +  
  geom_bar(position = "dodge") +  
  labs(title = "Installment Rate by Risk",  
        x = "Installment rate category",  
        y = "Count") +  
  theme_minimal()
```

Category 4 contains the largest number of applicants and therefore also has the largest number of Good and Bad cases in absolute terms. However, this figure shows **counts**, not risk proportions, so we cannot conclude that category 4 is “riskier” without computing the **Bad percentage within each category**. This chart mainly helps visualize how the dataset is distributed across categories.

Part 2 — KNN Modeling (training, tuning, evaluation)

```
> if (length(to_install) > 0) install.packages(to_install)
```

```
Erreur : objet 'to_install' introuvable
```

```
> invisible(lapply(pkgs, library, character.only = TRUE))
```

```
— Attaching core tidyverse packages — tidyverse
```

```
✓ dplyr      1.1.4      ✓ readr      2.1.6
```

```
✓ forcats    1.0.1      ✓ stringr    1.6.0
```

```
✓ ggplot2    4.0.2      ✓ tibble     3.3.0
```

```
✓ lubridate  1.9.4      ✓ tidyr      1.3.2
```

```
✓ purrr      1.2.1
```

```
— Conflicts — tidyverse_conflict
```

```
✗ dplyr::filter() masks stats::filter()
```

```
✗ dplyr::lag() masks stats::lag()
```

```
i Use the conflicted package to force all conflicts to become errors
```

```
Attachement du package : 'recipes'
```

```
L'objet suivant est masqué depuis 'package:stringr':
```

```
fixed
```

```
L'objet suivant est masqué depuis 'package:stats':
```

```
step
```

```
Le chargement a nécessité le package : lattice
```

```
Attachement du package : 'caret'
```

```
L'objet suivant est masqué depuis 'package:rsample':
```

```
calibration
```

```
L'objet suivant est masqué depuis 'package:purrr':
```

```

> split <- initial_split(df, prop = 0.8, strata = Risk)
> train_df <- training(split)
> test_df <- testing(split)
> split <- initial_split(df, prop = 0.8, strata = Risk)
> train_df <- training(split)
> test_df <- testing(split)
> rec <- recipe(Risk ~ ., data = train_df) %>%
+   step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
+   step_zv(all_predictors()) %>%
+   step_normalize(all_numeric_predictors())
> prep_rec <- prep(rec, training = train_df)
> train_baked <- bake(prep_rec, new_data = train_df)
> test_baked <- bake(prep_rec, new_data = test_df)
> x_train <- train_baked %>% select(-Risk)
> y_train <- train_baked$Risk
> x_test <- test_baked %>% select(-Risk)
> y_test <- test_baked$Risk
> ctrl <- trainControl(
+   method = "repeatedcv",
+   number = 5,
+   repeats = 2,
+   classProbs = TRUE,
+   summaryFunction = twoClassSummary
+ )
> grid <- expand_grid(k = seq(3, 31, by = 2))
> knn_fit <- train(
+   x = x_train,
+   y = y_train,
+   method = "knn",
+   metric = "ROC",
+   tuneGrid = grid,
+   trControl = ctrl
+ )

> cat("\nBest k:\n")

Best k:
> print(knn_fit$bestTune)
      k
7 15
> cat("\nCV results (top rows):\n")

CV results (top rows):
> print(head(knn_fit$results[order(-knn_fit$results$ROC), ], 10))
      k      ROC      Sens      Spec      ROCSD      SensSD      SpecSD
7  15 0.7373698 0.9553571 0.1604167 0.03951358 0.017857143 0.03809963
6  13 0.7366257 0.9535714 0.1770833 0.04561568 0.023735002 0.04527224
15 31 0.7364769 0.9875000 0.0812500 0.02711397 0.011293849 0.03733244
12 25 0.7362723 0.9830357 0.1020833 0.03053959 0.011488249 0.04102515
5  11 0.7343750 0.9446429 0.1979167 0.04817437 0.016192227 0.03437325
8  17 0.7341425 0.9598214 0.1479167 0.03750152 0.015320878 0.04653296
10 21 0.7334356 0.9714286 0.1333333 0.03633280 0.009221389 0.06226809
13 27 0.7334077 0.9875000 0.0937500 0.03139551 0.010480248 0.04527224
14 29 0.7331752 0.9875000 0.0875000 0.03043032 0.010480248 0.05178153
11 23 0.7327567 0.9750000 0.1104167 0.03634710 0.010136540 0.04286472
> pred_class <- predict(knn_fit, newdata = x_test)
> pred_prob <- predict(knn_fit, newdata = x_test, type = "prob")

```

```
> cat("\nConfusion matrix (Bad as positive class):\n")
```

Confusion matrix (Bad as positive class):

```
> cm <- confusionMatrix(pred_class, y_test, positive = "Bad")
> print(cm)
```

Confusion Matrix and Statistics

	Reference	
Prediction	Good	Bad
Good	129	44
Bad	11	16

Accuracy : 0.725
95% CI : (0.6576, 0.7856)
No Information Rate : 0.7
P-Value [Acc > NIR] : 0.2455

Kappa : 0.2232

Mcnemar's Test P-Value : 1.597e-05

Sensitivity : 0.2667
Specificity : 0.9214
Pos Pred Value : 0.5926
Neg Pred Value : 0.7457
Prevalence : 0.3000
Detection Rate : 0.0800
Detection Prevalence : 0.1350
Balanced Accuracy : 0.5940

'Positive' Class : Bad

```
> roc_obj <- pROC::roc(response = y_test, predictor = pred_prob$Bad,
+                      levels = c("Good", "Bad"), direction = "<")
> cat("\nTest AUC:\n")
```

Test AUC:

```
> print(as.numeric(pROC::auc(roc_obj)))
[1] 0.7677381
```

```
> # 10) Cost evaluation (German credit common cost) -----
> # FP = Good predicted Bad (rejecting a good applicant) cost 1
> # FN = Bad predicted Good (accepting a bad applicant) cost 5
> tab <- table(Actual = y_test, Pred = pred_class)
> # Safely extract FN/FP
> FN <- if ("Bad" %in% rownames(tab) && "Good" %in% colnames(tab)) tab["Bad", "Good"] else 0
> FP <- if ("Good" %in% rownames(tab) && "Bad" %in% colnames(tab)) tab["Good", "Bad"] else 0
> cost_total <- 5 * FN + 1 * FP
> cost_avg <- cost_total / nrow(test_df)
> cat("\nCost summary (FN*5 + FP*1):\n")
```

Cost summary (FN*5 + FP*1):

```
> cat("FN (Bad->Good) =", FN, "\n")
FN (Bad->Good) = 44
> cat("FP (Good->Bad) =", FP, "\n")
FP (Good->Bad) = 11
> cat("Total cost      =", cost_total, "\n")
Total cost      = 231
> cat("Avg cost/obs    =", cost_avg, "\n")
Avg cost/obs    = 1.155
```

```

> library(class)
> ks <- seq(3, 31, by = 2)
> results <- do.call(rbind, lapply(ks, function(k) {
+   pred <- knn(train = x_train, test = x_test, cl = y_train, k = k)
+   cm <- table(Actual = y_test, Pred = pred)
+   accuracy <- sum(diag(cm)) / sum(cm)
+   FN <- cm["Bad", "Good"] # actual Bad predicted Good
+   FP <- cm["Good", "Bad"] # actual Good predicted Bad
+   data.frame(k = k, accuracy = accuracy, FN = FN, FP = FP)
+ })))
> results
  k accuracy FN FP
1  3    0.700 39 21
2  5    0.705 37 22
3  7    0.705 39 20
4  9    0.735 39 14
5 11    0.730 43 11
6 13    0.720 44 12
7 15    0.725 44 11
8 17    0.725 45 10
9 19    0.710 48 10
10 21    0.715 48  9
11 23    0.700 50 10
12 25    0.695 51 10
13 27    0.705 52  7
14 29    0.690 53  9
15 31    0.695 54  7

```

Insights and Discussion

1) Final model configuration

After preprocessing (one-hot encoding for categorical variables and normalization of numeric variables), we trained and tuned a K-Nearest Neighbors (KNN) classifier using cross-validation. The best-performing configuration selected by the tuning procedure was:

- **Best k = 15**

This means that each applicant is classified based on the majority class among their **15 nearest neighbors** in the training data.

2) Overall performance: accuracy is acceptable, but not sufficient

On the test set, the model reached:

- **Accuracy = 0.725 (72.5%)**
- **No Information Rate (baseline) = 0.70**

So, the model performs slightly better than the “always predict Good” baseline, but the improvement is not strong statistically (**p-value [Acc > NIR] = 0.2455**). In other words, accuracy alone suggests only a modest gain over a naive strategy.

Also, **Kappa = 0.2232**, which indicates only **fair** agreement beyond chance — again suggesting that the model is not strongly reliable overall.

3) Confusion matrix interpretation (credit-risk meaning)

Your confusion matrix (Positive class = **Bad**) is:

- **True Negatives (Good → Good) = 129**
- **False Positives (Good → Bad) = 11**
- **False Negatives (Bad → Good) = 44**
- **True Positives (Bad → Bad) = 16**

What this means in practice:

- The model is very good at recognizing **Good** clients (many TN).
- But it misses many **Bad** clients (high FN), which is the most dangerous error for a bank.

4) The most important insight: the model struggles to catch “Bad” applicants

Key class-based metrics:

- **Recall / Sensitivity (Bad) = 0.2667** → the model detects only **26.7%** of bad applicants

- **Specificity (Good)** = 0.9214 → it correctly identifies **92.1%** of good applicants
- **Precision (Bad)** = 0.5926 → when it predicts Bad, it is correct about **59.3%** of the time
- **Balanced Accuracy** = 0.5940

Interpretation:

The model is heavily better at identifying the **Good** class than the **Bad** class. This is a typical effect in imbalanced datasets (70/30): the classifier can obtain decent accuracy by predicting “Good” frequently, but this leads to many missed risky applicants.

5) Error asymmetry is confirmed (McNemar test)

The **McNemar test p-value** = **1.597e-05**, which indicates the model’s errors are not symmetric: it makes **significantly more FN than FP**.

This is important: the classifier is *systematically* more likely to label applicants as **Good**, which increases the risk of approving bad clients.

6) Cost-sensitive view (business impact)

In credit risk, approving a risky client is usually more expensive than rejecting a safe client. Using the common cost scheme:

- FP (Good → Bad) cost = 1
- FN (Bad → Good) cost = 5

Your results give:

- **FN = 44, FP = 11**
- **Total cost** = $5 \times 44 + 1 \times 11 = 231$
- Test size = 200 → **Average cost per applicant** = $231 / 200 = 1.155$

Business meaning:

Most of the cost comes from **Bad applicants predicted as Good** (44 cases). So even though accuracy looks okay, the model could still generate high losses if used directly for approval decisions.

7) Why k = 15 led to this behavior

With **k = 15**, the voting tends to favor the majority class (“Good”), especially when the neighborhood contains many Good examples. This stabilizes predictions and improves specificity (Good detection), but it reduces sensitivity for Bad (risk detection). This explains why the model is strong at predicting Good but weak at catching Bad.

Conclusion

This project applied a K-Nearest Neighbors (KNN) approach to the German Credit dataset in order to classify loan applicants as **Good** or **Bad** credit risk. After preparing the dataset (renaming variables, converting the target to Good/Bad), and performing exploratory analysis, we built a complete KNN pipeline including train/test splitting, one-hot encoding of categorical variables, and normalization of numeric variables to ensure that distance calculations were meaningful.

The final tuned KNN model selected **$k = 15$** and achieved an **accuracy of 72.5%** on the test set, with an **AUC of approximately 0.7677**, indicating a moderate ability to distinguish between Good and Bad applicants. However, the confusion matrix highlights an important limitation: the model's **sensitivity (recall) for the Bad class is only 26.7%**, meaning that it correctly identifies only about one quarter of risky applicants. In contrast, the model shows a very high **specificity of 92.1%** for Good applicants. In business terms, the model tends to predict "Good" often, which results in many **false negatives (Bad predicted as Good)**—a critical issue for credit risk, since approving a risky applicant can generate significant losses.

Overall, KNN provides a useful baseline and demonstrates reasonable overall performance, but it is not yet reliable for a decision process that prioritizes identifying risky clients. Future improvements should focus on reducing false negatives and improving Bad detection, for example by using **class balancing techniques, weighted KNN**, or by comparing with other models commonly used for credit scoring such as **logistic regression, random forests, or gradient boosting**.